

Trabajo Practico N° 3

1. Shell Scripting.

El Shell Scripting es la práctica de escribir scripts, es decir, archivos de texto con instrucciones que son interpretados y ejecutados por un shell, como Bash, Zsh, Sh, entre otros. Un shell es un intérprete de comandos en sistemas Unix/Linux que permite interactuar con el sistema operativo. Es práctico para manejar archivos, y es extremadamente útil y simple para crear procesos y manipular sus salidas.

Los shell scripts se usan principalmente para:

- Automatización de tareas repetitivas
- Administración del sistema
- Procesamiento de archivos y datos
- Orquestación de comandos

Los scripts NO necesitan compilarse, los shell scripts son interpretados, no compilados. El shell lee el script línea por línea y ejecuta cada instrucción directamente. Un shell es un intérprete, no un compilador.

Un compilador es el que se encarga de traducir el programa completo a código ejecutable antes de correrlo, si hay un error se detecta en tiempo de compilación. Un intérprete ejecuta el código directamente, instrucción por instrucción, y si hay un error se detecta en tiempo de ejecución.

2. Comandos echo y read. Comentarios y variables.

echo

```
demianbz@debian:~$ echo "Hola Mundo"
Hola Mundo
demianbz@debian:~$ echo "El usuario actual es: $USER"
El usuario actual es: demianbz
```

read

```
demianbz@debian:~$ read nombre
pepe
demianbz@debian:~$ echo "Hola mi nombre es $nombre"
Hola mi nombre es pepe
demianbz@debian:~$ read -p "Ingrese su nombre: " nombre
Ingrese su nombre: pepep
demianbz@debian:~$ echo "Hola mi nombre es $nombre"
Hola mi nombre es pepep
```

comentarios

```
demianbz@debian:~$ echo "Hola" #Los comentarios se indican con el signo "#"
```

variables

```
demianbz@debian:~$ nombre="Carlos" edad=25
demianbz@debian:~$ echo "Nombre: $nombre , Edad: $edad"
Nombre: Carlos , Edad: 25
demianbz@debian:~$ echo "Hola ${nombre}" #Se utiliza llave para evitar ambigüedades
Hola Carlos
```

3. Ejercicio 3

Creo el directorio "practica-shell-script" (previamente me voy posicionando con cd para ir viendo los directorios)

```
demianbz@debian:/$ cd /
demianbz@debian:/$ ls
bin  dev  home  initrd.img.old  lib64  media  opt  root  sbin  sys  usr  vmlinuz
boot  etc  initrd.img  lib  lost+found  mnt  proc  run  srv  tmp  var  vmlinuz.old
demianbz@debian:/$ cd home
demianbz@debian:/home$ ls
demianbz
demianbz@debian:/home$ cd demianbz
demianbz@debian:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
demianbz@debian:~$ mkdir practica-shell-script
demianbz@debian:~$ ls
Desktop  Documents  Downloads  Music  Pictures  practica-shell-script  Public  Templates  Videos
```

Creo el archivo llamado "mostrar.sh"

```
demianbz@debian:~$ cd practica-shell-script
demianbz@debian:~/practica-shell-script$ touch mostrar.sh
demianbz@debian:~/practica-shell-script$ ls
mostrar.sh
```

Edito el archivo con el contenido pedido (nano mostrar.sh)

```
GNU nano 8.4                                mostrar.sh
#!/bin/bash
#Comentarios acerca de lo que hace el script
#Siempre comento mis scripts, si no lo hago hoy,
#Mañana ya no me acuerdo de lo que quise hacer
echo "Introduzca su nombre y apellido"
read nombre apellido
echo "Fecha y hora actual:"
date
echo "Su apellido y nombre es:"
echo "$apellido $nombre"
echo "Su usuario es:`whoami`"
echo "Su directorio actual es:"
echo `pwd`
```

Cambio los permisos para que el usuario pueda ejecutar el archivo. (chmod 754).

```
demianbz@debian:~/practica-shell-script$ ls -l mostrar.sh
-rw-rw-r-- 1 demianbz demianbz 374 Jun 16 22:54 mostrar.sh
demianbz@debian:~/practica-shell-script$ chmod 754 mostrar.sh
demianbz@debian:~/practica-shell-script$ ls -l mostrar.sh
-rwxrwx--w- 1 demianbz demianbz 374 Jun 16 22:54 mostrar.sh
```

Ejecuto el archivo (./mostrar.sh)

```
demianbz@debian:~/practica-shell-script$ ./mostrar.sh
Introduzca su nombre y apellido
Demian Bonzi
Fecha y hora actual:
Wed Jun 17 03:10:29 PM UTC 2026
Su apellido y nombre es:
Bonzi Demian
Su usuario es:demianbz
Su directorio actual es:
/home/demianbz/practica-shell-script
```

Realizo modificaciones en el archivo para agregar funciones

```
GNU nano 8.4                                mostrar.sh
#!/bin/bash
#Comentarios acerca de lo que hace el script
#Siempre comento mis scripts, si no lo hago hoy,
#Mañana ya no me acuerdo de lo que quise hacer
echo "Introduzca su nombre y apellido"
read nombre apellido
echo "Fecha y hora actual:"
date
echo "Su apellido y nombre es:"
echo "$apellido $nombre"
echo "Su usuario es:`whoami`"
echo "Su directorio actual es:"
echo `pwd`
echo "Su directorio personal es: $HOME"
echo "Ingrese un directorio para mostrar su contenido: "
read dir
ls "$dir"
echo "Espacio libre en el disco: `df`"
```

Resultado:

```
demianbz@debian:~/practica-shell-script$ ./mostrar.sh
Introduzca su nombre y apellido
Demian Bonzi
Fecha y hora actual:
Wed Jun 17 03:25:22 PM UTC 2026
Su apellido y nombre es:
Bonzi Demian
Su usuario es:demianbz
Su directorio actual es:
/home/demianbz/practica-shell-script
Su directorio personal es: /home/demianbz
Ingrese un directorio para mostrar su contenido:
/home
demianbz
Espacio libre en el disco: Filesystem      1K-blocks    Used Available Use% Mounted on
udev                973216         0    973216     0% /dev
tmpfs               202076      1192    200884     1% /run
/dev/sda1          19353424 5072028 13272964   28% /
tmpfs              1010376        12   1010364     1% /dev/shm
tmpfs              1024          0     1024     0% /run/credentials/systemd-journald.service
tmpfs              5120          8     5112     1% /run/lock
tmpfs              1010380       16   1010364     1% /tmp
tmpfs              202072       96    201976     1% /run/user/1000
```

Para que el comando df (que es el que muestra el espacio disponible en el disco) muestre los datos de una forma mas legible se utiliza "df -h" que es human-readable.

Ejemplo:

```
Espacio libre en el disco: Filesystem      Size  Used Avail Use% Mounted on
udev                951M   0    951M   0% /dev
tmpfs              198M  1.2M  197M   1% /run
/dev/sda1          19G  4.9G   13G  28% /
tmpfs              987M  12K   987M   1% /dev/shm
tmpfs              1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
tmpfs              5.0M  8.0K   5.0M   1% /run/lock
tmpfs              987M  16K   987M   1% /tmp
tmpfs              198M  96K   198M   1% /run/user/1000
```

4. Parametrización.

Cuando se ejecuta un script como:

```
./mi_script.sh arg1 arg2 arg3
```

Dentro del script se puede acceder a cada parámetro posicional usando:

\$0 → el nombre del script

\$1 → primer argumento (arg1)

\$2 → segundo argumento (arg2)

\$3 → tercer argumento (arg3) ... y así sucesivamente.

\$# - Numero de parámetros: Devuelve cuantos argumentos fueron pasados al script.

\$* - Todos los parámetros como una sola cadena: Contiene todos los argumentos, expandidos como una sola palabra.

\$? - Código de salida del ultimo comando: Muestra si el ultimo comando se ejecutó correctamente.

0 -> éxito

cualquier otro número -> hubo error

\$HOME - Directorio home del usuario: Esta variable de entorno contiene la ruta del directorio personal del usuario actual.

5. Comando exit.

El comando **exit** en un script de Shell (**bash**) sirve para **finalizar la ejecución** del script o de una función, y opcionalmente devolver un código de salida al sistema. Detiene inmediatamente la ejecución del script y devuelve un valor numérico al SO. Exit puede recibir un numero entero entre 0 y 255:

exit N

N es el código de salida.

0 -> Éxito todo salió correctamente.

1-255 -> Algún tipo de error

0 -> Ejecución exitosa

1 -> Error genérico

2 -> Error de uso incorrecto del Shell o de los argumentos

126 -> Comando encontrado pero no ejecutable

127 -> Comando no encontrado

130 -> Script terminado con Ctrl + C (SIGINT)

6. Comando expr.

El comando **expr** en **bash** permite evaluar expresiones aritméticas, relacionales, lógicas y de manipulación de cadenas.

- Operaciones aritméticas:

Permite realizar cálculos numéricos.

+ Suma	expr 4 + 5 → 9
- Resta	expr 10 - 3 → 7
* Multiplicación	expr 4 * 5 → 20
/ División entera	expr 7 / 2 → 3
% Módulo	expr 10 % 3 → 1

- Operaciones relacionales:

Usadas para comparar valores (retornan 1 = verdadero, 0 = falso).

= Igual	expr 5 = 5 → 1
\!= Distinto	expr 5 \!= 3 → 1
> Mayor que	expr 7 > 2 → 1
< Menor que	expr 2 < 1 → 0
>= Mayor o igual	expr 3 >= 3 → 1
<= Menor o igual	expr 1 <= 5 → 1

- Operaciones con cadenas:

Permite trabajar con texto.

▪ Comparación de cadenas

expr "hola" = "hola"

▪ Longitud de una cadena

expr length "texto" → 5

▪ Substring (extracto)

expr substr "computadora" 2 4 → "ompu" (Posición inicial: 1)

▪ Índice de un carácter dentro de una cadena

expr index "banana" a

7. Comando test expresión.

El comando **test** permite evaluar:

- Expresiones de archivos.
- Expresiones de cadenas de texto.
- Expresiones numéricas.

El resultado será: 0 (true) si la expresión es verdadera. 1 (false) si es falsa.

• Evaluación de archivos

Sirven para comprobar la existencia, tipo o permisos de un archivo/directorio.

Comprobación de existencia y tipo:

-e archivo El archivo existe (**exists**)

-f archivo	Existe y es un archivo regular (file)
-d archivo	Es un directorio (directory)
-l archivo	Es un enlace simbólico (link)
-s archivo	Existe y tiene tamaño mayor a cero (size)

Permisos:

-r archivo	Tiene permiso de lectura (read)
-w archivo	Tiene permiso de escritura (write)
-x archivo	Tiene permiso de ejecución (execute)

Comparación entre archivos:

archivo1 -nt archivo2	archivo1 es más nuevo (newer than)
archivo1 -ot archivo2	archivo1 es más viejo (older than)
archivo1 -ef archivo2	Son el mismo archivo (equal file)

• **Evaluación de cadenas de caracteres**

Longitud:

-z cadena	Longitud cero (cadena vacía) (zero)
-n cadena	Longitud mayor a cero (non-cero)

Comparación de cadenas:

cadena1 = cadena2	Son iguales
cadena1 != cadena2	Son distintas

• **Evaluaciones numéricas**

Se usan operadores específicos para enteros.

-eq	Igual que (equal)
-ne	Distinto (not equal)
-gt	Mayor que (greater than)
-lt	Menor que (less than)
-ge	Mayor o igual que (greater or equal)
-le	Menor o igual que (less or equal)

• **Combinar expresiones**

Se pueden combinar varias condiciones

AND (y)	[condicion1 -a condicion2]
OR (o)	[condicion1 -o condicion2]
NOT (negación)	[! condición]

8. Estructuras de control

Selección de alternativas: (if y case)

Decisión:	Selección:
<pre>if [condition] then bloque elif [condition] then bloque else bloque fi</pre>	<pre>case \$variable in "valor 1") bloque ;; "valor 2") bloque ;; *) bloque ;; esac</pre>

Menú de opciones: (select)

Ejemplo:

```
select accion in Nuevo Salir  
do  
  case $accion in  
    "Nuevo")  
      echo "Seleccionado: Nuevo"  
      ;;  
    "Salir")  
      exit 0  
      ;;  
  esac  
done
```

Iteración: (for)

C-style:

```
for ((i=0; i < 10; i++))  
do  
  bloque  
done
```

Con lista de valores (foreach):

```
for i in valor1 valor2 valor3 valorN  
do  
  bloque  
done
```

Iteración (while)

```
while [ condicion ] # Mientras se cumpla la condición  
do  
  bloque  
done
```

9. Sentencia break y continue

Dentro de un bucle, las sentencias break y continue **se utilizan para alterar el flujo normal de ejecución**. Pueden recibir parámetros cuando están en bucles anidados.

break

Interrumpe completamente el bucle actual en el que se encuentra. Si recibe un número como parámetro, sale ese número de niveles de bucles anidados.

break n -> rompe n niveles

continue

Salta al inicio de la siguiente iteración. Si está dentro de varios bucles anidados, puede saltar al inicio del bucle de N niveles hacia afuera.

continue 1 -> afecta al bucle actual

continue 2 -> vuelve al siguiente ciclo del segundo bucle externo

10. Variables y arreglos

Declaración de variables

En Bash, se declaran simplemente asignando un valor sin espacios:

```
nombre="Carlos"
```

```
edad=25
```

Referencia a variables

Para usar una variable, se antepone el símbolo \$:

```
echo "Nombre: $nombre"
```

```
echo "Edad: $edad"
```

También se pueden usar llaves para evitar ambigüedades:

```
echo "Hola ${nombre}"
```

En bash, las variables no tienen tipo de dato fijo. Todo se maneja como cadenas de texto, (bash permite interpretarlas como números en contextos aritméticos).

Tipos según uso:

Variables de texto (por defecto todo es texto)

- Variables numéricas
- Variables de entorno
- Variables especiales (ej.: \$?, \$0, \$1, \$#, etc.)
- Arreglos (arrays indexados)

Bash No es un lenguaje fuertemente tipado. **No se necesita declarar el tipo de una variable**. Se puede asignar números, textos, rutas, etc.

Arreglos

- Creación:

```
arreglo_a=() # Se crea vacío
```

```
arreglo_b=(1 2 3 5 8 13 21) # Inicializado
```

- Asignación de un valor en una posición concreta:

```
arreglo_b[2]=spam
```


- Acceso a un valor del arreglo (en este caso las llaves no son opcionales):

```
echo ${arreglo_b[2]}  
copia=${arreglo_b[2]}
```

- Acceso a todos los valores del arreglo:

```
echo ${arreglo[@]}          # o bien ${arreglo[*]}
```

- Tamaño del arreglo:

```
${#arreglo[@]}             # otra forma ${#arreglo[*]}
```

- Borrado de un elemento (reduce el tamaño del arreglo, pero no elimina la posición, solamente la deja vacía):

```
unset arreglo[2]
```

- Los índices en los arreglos comienzan en 0

11. Funciones

Las funciones permiten modularizar el comportamiento de los scripts.

- Se pueden declarar de 2 formas:

- **function nombre { bloque }**
- **nombre() { bloque }**

- Con la sentencia return se retorna un valor entre 0 y 255
- El valor de retorno se puede evaluar mediante la variable \$?
- Reciben argumentos en las variables \$1, \$2, etc.

Los parámetros de una función no se declaran, sino que se reciben igual que los parámetros de un script.

\$1 -> primer argumento.
\$2 -> segundo argumento.
\$@ -> todos los argumentos.
\$# -> cantidad de argumentos.

Ejemplo:

```
saludar() {  
    echo "Hola $1, tu edad es $2"  
}  
saludar "Juan" 25
```

Salida: Hola Juan, tu edad es 25

12. Evaluación de expresiones

Creo y edito el archivo con nano:

```
GNU nano 8.4                                ejer12
#!/bin/bash

echo "Ingrese 2 numeros:"
read num1 num2
echo "$num1 + $num2 = `expr $num1 + $num2`"
echo "$num1 - $num2 = `expr $num1 - $num2`"
echo "$num1 x $num2 = `expr $num1 \* $num2`"

echo "El mayor de los dos numeros ingresados es:"
if [ $num1 -gt $num2 ]
then
    echo $num1
elif [ $num1 -lt $num2 ]
then
    echo $num2
else
    echo "los numeros son iguales"
fi
```

Lo ejecuto:

```
demianbz@debian:~/practica-shell-script$ ./ejer12
Ingrese 2 numeros:
6 3
6 + 3 = 9
6 - 3 = 3
6 x 3 = 18
El mayor de los dos numeros ingresados es:
6
```

Inciso b)

Creo y edito el archivo con nano:

```
GNU nano 8.4                                ejer12b
#!/bin/bash

if [ $# -ne 2 ]
then
    echo "No se ingresaron la cantindad correcta de parametros"
    exit 1
fi

#Asigno los parametros
num1=$1
num2=$2

echo "$num1 + $num2 = `expr $num1 + $num2`"
echo "$num1 - $num2 = `expr $num1 - $num2`"
echo "$num1 x $num2 = `expr $num1 \* $num2`"

echo "El mayor de los dos numeros ingresados es:"
if [ $num1 -gt $num2 ]
then
    echo $num1
elif [ $num1 -lt $num2 ]
then
    echo $num2
else
    echo "los numeros son iguales"
fi
```

Lo ejecuto:

```
demianbz@debian:~/practica-shell-script$ ./ejer12b 5 5
5 + 5 = 10
5 - 5 = 0
5 x 5 = 25
El mayor de los dos numeros ingresados es:
los numeros son iguales
```

Inciso c)

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer12c
if [ $# -ne 3 ]
then
    echo "No se ingresaron la cantidad correcta de parametros"
    echo "Ingrese 3 parametros num1 num2 operacion"
    exit 1
fi

#Asigno los parametros
num1=$1
num2=$2
operacion=$3

case $operacion in
    "+")
        echo "$num1 + $num2 = `expr $num1 + $num2`"
        ;;
    "-")
        echo "$num1 - $num2 = `expr $num1 - $num2`"
        ;;
    "*")
        echo "$num1 x $num2 = `expr $num1 \* $num2`"
        ;;
    "/")
        echo "$num1 / $num2 = `expr $num1 / $num2`"
        ;;
esac
```

Salida:

```
demianbz@debian:~/practica-shell-script$ ./ejer12c 10 5 /
10 / 5 = 2
```

13. Estructuras de control.

Inciso a)

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer13a
#!/bin/bash

for ((i=1; i<101; i++))
do
    echo "$i - $i^2= `expr $i \* $i`"
done
```

Inciso b)

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer13b
#!/bin/bash

echo "----Elija el numero de una de las opciones----"
echo "----1.Listar | 2.DondeEstoy | 3.QuienEsta----"
read opcion

case $opcion in
    "1")
        echo "Contenido del directorio actual:"
        echo `ls`
        ;;
    "2")
        echo "Ruta del directorio donde se encuentra:"
        echo `pwd`
        ;;
    "3")
        echo "Usuarios conectados al sistema"
        echo `who`
        ;;
    *)
        echo "Opcion no valida"
        exit 1
esac
```

Inciso c)

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer13c

if [ $# -ne 1 ]
then
    echo "Error en la cantidad de parametros, debe ingresar 1 parametro"
    exit 1
fi

archivo=$1

if [ -e "$archivo" ]
then
    if [ -f "$archivo" ]
    then
        echo "Existe y es un archivo"
    elif [ -d "$archivo" ]
    then
        echo "Existe y es un directorio"
    else
        echo "Existe pero no es ni un archivo ni un directorio"
    fi
else
    echo "El archivo no existe"
    mkdir "$archivo"
    echo "Se creo correctamente el directorio $archivo"
fi
```

Tengo que usar "\$variable" en lugar de \$variable, porque las comillas evitan que bash divida el contenido de la variable en varios argumentos.

Ejemplo:

archivo="Mi carpeta"

mkdir \$archivo

bash interpreta dos directorios Mi y Carpeta
con comillas ("\$variable") bash interpreta "Mi Carpeta"

14. Renombrar archivos.

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer14 *

if [ $# -ne 3 ]
then
    echo "No se ingresaron la cantidad de parametros adecuada, debe ingresar 3 parametros"
    exit 1
fi
directorio=$1
opcion=$2
cadena=$3
if [ -d "$directorio" ]
then
    if [ "$opcion" = "-a" ]
    then
        for archivo in "$directorio"/*
        do
            if [ -f "$archivo" ]
            then
                mv "$archivo" "$archivo$cadena"
            fi
        done
    elif [ "$opcion" = "-b" ]
    then
        for archivo in "$directorio"/*
        do
            if [ -f "$archivo" ]
            then
                nombreArchivo=$(basename "$archivo")
                mv "$archivo" "$directorio/$cadena$nombreArchivo"
            fi
        done
    else
        echo "La opcion ingresada no es correcta, ingrese -a o -b"
    fi
else
    echo "El directorio ingresado no es valido"
fi
```

```

demianbz@debian:~/practica-shell-script$ ./ejer14 directorioEjer13 -a EJ
demianbz@debian:~/practica-shell-script$ cd directorioEjer13
demianbz@debian:~/practica-shell-script/directorioEjer13$ ls
pepeEJ  prueba1EJ  prueba2EJ
demianbz@debian:~/practica-shell-script$ ./ejer14 directorioEjer13 -b EJ
demianbz@debian:~/practica-shell-script$ cd directorioEjer13
demianbz@debian:~/practica-shell-script/directorioEjer13$ ls
EJpepeEJ  EJprueba1EJ  EJprueba2EJ

```

15. Comando cut.

El comando cut permite extraer secciones de líneas de un archivo o de la entrada estándar. Puede cortar por posición (bytes o caracteres) o por campos delimitados.

Parámetros principales de cut:

1. -b LIST

Corta bytes específicos de cada línea.

Ejemplo:

Archivo 'ABCDEFGG' llamado ejemplo.txt.

```
cut -b 1-3 ejemplo.txt -> retorna ABC
```

2. -c LIST

Corta caracteres específicos.

Ejemplo:

Archivo 'computadora' llamado ejemplo.txt.

```
cut -c 5-8 ejemplo.txt -> retorna utad-
```

3. -f LIST

Selecciona campos específicos delimitados por un separador.

Ejemplo:

Archivo llamado datos.csv:

Juan,25,Argentina

Maria,30, Uruguay

```
cut -d ',' -f 1 datos.csv -> retorna Juan
Maria
```

-d indica por cual carácter están separados los campos, y -f indica que campos extraer, en este caso los campos están separados por ',' y elijo el campo 1 (el nombre).

4. -d DELIM

Selecciona el delimitador cuando se usa con -f.

Ejemplo:

Archivo llamado info.txt:

123;abc;999

456;def;888

```
cut -d ';' -f 2 info.txt -> retorna abc
def
```

5. --complement

Invierte la selección (muestra todo menos lo seleccionado).

Ejemplo:

Archivo llamado datos.csv:

Juan,25,Argentina

Maria,30,Uruguay

```
cut -d ',' -f 2 --complement personas.csv -> retorna Juan,Argentina
Maria,Uruguay
```

6. -s

Suprime líneas que no contengan el delimitador.

Ejemplo:

Archivo llamado mix.txt:

hola mundo

uno;dos;tres

sin delimitador

a;b;c

```
cut -d ';' -f 2 -s mix.txt -> retorna dos
b
```

solo procesa líneas que tienen como delimitador ';'

16. Script Reporte.

Creo y edito el archivo con nano

```
GNU nano 8.4                               ejer16
#!/bin/bash

if [ $# -ne 1 ];then
    echo "Error no se ingreso la extension como parametro"
    exit 10
fi

extension=$1

usuarios=$(cut -d ":" -f1 /etc/passwd)
> reporte.txt #Creo vacio el archivo de texto de reporte

for usuario in $usuarios ;do
    home=$(grep "^$usuario:" /etc/passwd | cut -d ":" -f6)
    if [ -d "$home" ] ;then
        cantArch=$(find "$home" -type f -name "$extension" 2>/dev/null | wc -l)
        echo "$usuario $cantArch" >> reporte.txt
    else
        echo "El directorio $home del usuario $usuario no existe"
    fi
done
```

17. Script letras invertidas

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer17
#!/bin/bash

for archivo in * ;do
    echo "$archivo" | tr 'A-Za-z' 'a-zA-Z' | tr -d 'aA'
done
```

18. Script verificación logeo.

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer17 *
#!/bin/bash

if [ $# -ne 1 ] ;then
    echo "Error no se ingreso un usuario como parametro"
    exit 10
fi

usuario=$1

while ! who | cut -d " " -f1 | grep "^$usuario$" ;do
    sleep 10
done

echo "El usuario $usuario logueado en el sistema"
```

19. Pila en bash.

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer19
#!/bin/bash

opcion=-1

while [ "$opcion" -ne 0 ] ;do
    echo "-----MENU DE COMANDOS-----"
    echo "12. Ejercicio 12"
    echo "13. Ejercicio 13"
    echo "14. Ejercicio 14"
    echo "15. Ejercicio 15"
    echo "16. Ejercicio 16"
    echo "17. Ejercicio 17"
    echo "18. Ejercicio 18"
    echo "0. Salir del menu"
    echo "-----"
    echo "Ingrese una opcion:"
    read opcion

    case $opcion in
        12)
            ./ej12
            ;;
        13)
            ./ej13
            ;;
        14)
            echo "Ingrese un directorio"
```

20. Pila en bash

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer20
#!/bin/bash

lista=()

push(){
    if [ $# -ne 1 ] ;then
        echo "Error no se ingresaron parametros"
        return 10
    fi

    elemento=$1
    lista+=("$elemento")
}

pop(){
    tam=${#lista[@]}
    ult=$(expr $tam - 1)
    unset lista[$ult]
}

length(){
    echo "${#lista[@]}"
}

print(){
    echo "${lista[@]}"
}

#Programa Principal
for ((i=0; i<10; i++)) ;do
    push "$i"
done

echo "Lista:"
print
echo "Tamaño:"
length

pop
pop
pop

echo "Lista:"
print
echo "Tamaño:"
length
```

21. Productoria

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer21
#!/bin/bash

arreglo=(10 3 5 7 9 3 5 4)

productoria(){
    resultado=1
    for ((i=0; i<${#arreglo[@]}; i++)) ;do
        resultado=$(expr $resultado \* ${arreglo[$i]})
    done
    echo "$resultado"
}

#Programa Principal

echo "arreglo: ${arreglo[@]}"
echo "Productoria del arreglo"
productoria
```


22. Pares e Impares

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer22
#!/bin/bash

arreglo=(10 3 5 7 9 3 5 4)
echo "arreglo: ${arreglo[@]}"
cantImpares=0

for ((i=0; i<${#arreglo[@]}; i++)) ;do
    if [ $(expr ${arreglo[$i]} % 2) -eq 0 ] ;then
        echo "Par: ${arreglo[$i]}"
    else
        cantImpares=$(expr $cantImpares + 1)
    fi
done

echo "Cantidad de impares: $cantImpares"
```

23. Suma de vectores

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer23
#!/bin/bash

vector1=(1 3 5 7 9)
vector2=(2 4 6 8 10)

for ((i=0; i<${#vector1[@]}; i++)) ;do
    suma=$(expr ${vector1[$i]} + ${vector2[$i]})
    echo "La suma de los elementos de la posicion $i de los vectores es $suma"
done
```

24. Arreglo de users

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer24
#!/bin/bash

arreglo=()

usuarios=$(cat /etc/group | grep "^users:" | cut -d ":" -f4)
usuarios=$(echo "$usuarios" | tr ' ' ',')

for usuario in $usuarios ;do
    arreglo+=("$usuario")
done

if [ $# -ne 0 ] ;then
    opcion=$1
    case $opcion in
        -b)
            if [ $# -ne 2 ] ;then
                echo "No se ingreso la posicion"
            else
                if [ "$2" -ge 0 ] && [ "$2" -lt "${#arreglo[@]}" ] ;then
                    echo "${arreglo[$2]}"
                else
                    echo "Error esa posicion no es valida"
                fi
            fi
        ;;
        -l)
            echo "Longitud del arreglo: ${#arreglo[@]}"
        ;;
        -i)
            echo "Arreglo: ${arreglo[@]}"
        ;;
        *)
            echo "Opcion invalida"
        esac
    else
        echo "Arreglo de usuarios creado"
    fi
```

25. Script informe directorio y archivos

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer25 *
#!/bin/bash

if [ $# -lt 1 ] ;then
    echo "Error debe ingresar al menos un paramtero"
    exit 10
fi

cantInexistentes=0
numParametro=1
for ruta in "$@" ;do
    if [ $(expr $numParametro % 2) -ne 0 ] ;then
        if [ -e "$ruta" ] ;then
            if [ -d "$ruta" ] ;then
                echo "Existe y es un directorio"
            fi
            if [ -f "$ruta" ] ;then
                echo "Existe y es un archivo"
            fi
        else
            echo "No existe"
            cantInexistentes=$(expr $cantInexistentes + 1)
        fi
    fi
    numParametro=$(expr $numParametro + 1)
done

echo "Cantidad de archivos inexistentes: $cantInexistentes"
```

26. Operaciones sobre arreglos

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer26
#!/bin/bash

array=()

inicializar(){
    array=()
}

agregar_elem(){
    if [ $# -ne 1 ] ;then
        echo "Error se debe ingresar un parametro"
        return 10
    fi
    array+=("$1")
}

eliminar_elem(){
    if [ $# -ne 1 ] ;then
        echo "Error se debe ingresar un parametro"
        return 10
    fi
    if [ $1 -ge 0 ] && [ $1 -lt ${#array[@]} ] ;then
        unset array[$1]
    fi
}

longitud(){
    echo "${#array[@]}"
}

imprimir(){
    echo "${array[@]}"
}

inicializar_Con_Valores(){
    if [ $# -ne 2 ] ;then
        echo "Error se deben ingresar dos parametros"
        return 10
    fi
    longitud=$1
    valor=$2
    array=()
    for ((i=0; i<$longitud; i++)) ;do
        array+=("$valor")
    done
}
```

27. Permisos de usuario

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer27

if [ $# -ne 1 ] ;then
    echo "Error se debe ingresar un parametro"
    exit 10
fi

if [ -d "$1" ] ;then
    lectura=0
    escritura=0
    directorio=$1
    for archivo in "$(find "$directorio" -type f)" ;do
        if [ -f "$archivo" ] ;then
            if [ -w "$archivo" ] ;then
                escritura=$((escritura + 1))
            fi
            if [ -r "$archivo" ] ;then
                lectura=$((lectura + 1))
            fi
        fi
    done
    echo "Cantidad de archivos con permiso de lectura: $lectura"
    echo "Cantidad de archivos con permiso de escritura: $escritura"
else
    exit 4
fi
```

28. Arreglo con archivos de /home

```
GNU nano 8.4                                ejer28 *
#!/bin/bash

arreglo=()

for archivo in $(find /home -type f -name "*.doc") ;do
    arreglo+=("$archivo")
done

verArchivo(){
    if [ $# -ne 1 ] ;then
        echo "Error se debe ingresar un parametro"
        return 10
    fi
    encuentre=0
    for ((i=0; i<${#arreglo[@]}; i++)) ;do
        nombreArchivo=$(basename "${arreglo[$i]}")
        if [ "$nombreArchivo" = "$1" ] ;then
            echo "Nombre del archivo: $nombreArchivo"
            encuentre=1
            break
        fi
    done
    if [ $encuentre -eq 0 ] ;then
        echo "Archivo no encontrado"
        return 5
    fi
}

cantidadArchivos(){
    echo "${#arreglo[@]}"
}

borrarArchivo(){
    if [ $# -ne 1 ] ;then
        echo "Error se debe ingresar un parametro"
        return 10
    fi
    encuentre=0
    ruta=""
    for ((i=0; i<${#arreglo[@]}; i++)) ;do
        nombreArchivo=$(basename "${arreglo[$i]}")
        if [ "$nombreArchivo" = "$1" ] ;then
            ruta=${arreglo[$i]}
            unset arreglo[$i]
            encuentre=1
            break
        fi
    done
    if [ $encuentre -eq 0 ] ;then
        echo "Archivo no encontrado"
        return 10
    fi

    echo "Desea eliminar el archivo logicamente 1(Si)/0(No):"
    read opcion
    if [ $opcion -eq 0 ] ;then
        rm "$ruta"
    fi
}
```

29. Script mover programas

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer29
#!/bin/bash

if [ ! -d "$HOME/bin" ] ;then
    mkdir "$HOME/bin"
fi

movidos=0

for archivo in * ;do
    if [ -f "$archivo" ] ;then
        if [ -x "$archivo" ] ;then
            mv "$archivo" "$HOME/bin"
            echo "se movio el archivo $archivo"
            movidos=$(expr $movidos + 1)
        fi
    fi
done

if [ $movidos -gt 0 ] ;then
    echo "Cantidad de archivos movidos: $movidos"
else
    echo "No se movio ningun archivo"
fi
```

30. Estructura de datos Set

Creo y edito el archivo con nano

```
GNU nano 8.4                                ejer30
#!/bin/bash
initialize(){
    set=()
}

initialize_with(){
    if [ $# -lt 1 ] ;then
        echo "Error debe ingresar al menos un parametro"
        return 10
    fi
    for valor in "$@" ;do
        if ! contains "$valor" ;then
            set+=("$valor")
        fi
    done
}

add(){
    if [ $# -ne 1 ] ;then
        echo "Error se debe ingresar un parametro"
        return 10
    fi
    contains "$1"
    existe=$?
    if [ $existe -eq 1 ] ;then
        set+=("$1")
    else
        echo "Error no se pudo agregar porque ya existe"
    fi
    return $existe
}

remove(){
    if [ $# -lt 1 ] ;then
        echo "Error se debe ingresar al menos un parametro"
        return 10
    fi
    elimine=1
    for valor in "$@" ;do
        if contains "$valor" ;then
            for ((i=0; i<${#set[@]}; i++)) ;do
                if [ "${set[i]}" = "$valor" ] ;then
                    unset set[i]
                    elimine=0
                    break
                fi
            done
        fi
    done
    return $elimine
}

contains(){
    if [ $# -ne 1 ] ;then
        echo "Error se debe ingresar un parametro"
    fi
}
```

```

        return 10
    fi

    contiene=1
    for ((i=0; i<${#set[@]}; i++)) ;do
        if [ "$1" = "${set[$i]}" ] ;then
            contiene=0
            break
        fi
    done

    return $contiene
}

print(){
    for ((i=0; i<${#set[@]}; i++)) ;do
        echo "${set[$i]}"
    done
}

print_sorted(){
    print | sort
}

```

bingo.sh

```

GNU nano 8.4 bingo.sh
#!/bin/bash

# Cargo las funciones del Set
source ./ejer30

# Inicializo el conjunto
initialize

# Valor máximo por defecto
max=99

# Si el usuario pasó un parámetro
if [ $# -eq 1 ] ; then
    if [ "$1" -le 0 ] || [ "$1" -gt 32767 ] ; then
        echo "Error: el valor máximo debe estar entre 1 y 32767"
        exit 10
    fi
    max=$1
fi

respuesta=""

while [ "$respuesta" != "BINGO" ] ; do

    # Generar un número que no haya salido
    while true ; do
        numero=$((expr $RANDOM % $(expr $max + 1)))

        contains "$numero"

        if [ $? -ne 0 ] ; then
            add "$numero"
            break
        fi
    done

    echo "Número: $numero"

    echo "Presione ENTER para seguir o escriba BINGO para terminar:"
    read respuesta

done

echo
echo "Números sorteados:"
print_sorted

```

31. Directorio con estructura (Brace Expansión)

Creo y edito el archivo con nano

```

GNU nano 8.4 ejer31
#!/bin/bash

if [ $# -lt 1 ] ;then
    echo "Error: debe ingresar al menos un usuario"
    exit 10
fi

for usuario in "$@" ;do

    home=$(grep "^$usuario:" /etc/passwd | cut -d ":" -f6)

    if [ -d "$home" ] ;then

        mkdir -p "$home/directorio_iso"

        mkdir -p "$home/directorio_iso"/{2025,2026}/{01..12}

        touch "$home/directorio_iso"/{2025,2026}/{01..12}/archivo.txt

        echo "Se creo la estructura para el usuario $usuario"

    else
        echo "El usuario $usuario no existe o no posee un directorio personal valido"
    fi
done

```